

Practical Session: Locomotion Software for Legged Robots in Simulation

Objectives

This practical session introduces the main components of a model-based locomotion software stack for a quadruped robot in simulation. There are several blocks/modules in a locomotion framework, we will do exercises separately on them and then put all the modules together and experience and do tests with the whole framework. The goal is to progressively build, tune, and test the following modules of the framework: inverse kinematics, state estimation, terrain estimation and a Model Predictive Control (MPC) that leverages a centroidal model.

Install the software

First repo: We encourage you to use a conda environment installation following these [README](#).

```
git clone -b feat/lesson_friday https://github.com/iit-DLSLab/Quadruped-PyMPC.git
```

Very important: This clone automatically uses the branch `feat/lesson_friday`.

Very important: Every time you open a new terminal, you need to activate the conda environment first, see the README!

Second repo: [EKF for Legged Robots](#), see it's README for installation instructions. You need to install it in a separate conda environment, see the README!

Exercise 1: Numerical Inverse Kinematics

Inverse kinematics maps desired swing-foot trajectories into desired joint positions or velocities, to pass them to the lower-level PD control of the robot.

A damped least-squares solution is:

$$\Delta \mathbf{q} = \left(\mathbf{J}^T \mathbf{J} + \lambda^2 \mathbf{I} \right)^{-1} \mathbf{J}^T \mathbf{e}.$$

where $\mathbf{J}$ is the leg Jacobian, $\mathbf{e}$ is the Cartesian error, and λ is a damping factor. The joint reference can then be updated as:

$$\mathbf{q}_{k+1}^{\text{des}} = \mathbf{q}_k^{\text{des}} + \Delta \mathbf{q} \Delta t.$$

Hint: Stack all the error and jacobians together!

Starting file: `Quadruped-PyMPC/inverse_kinematics_numeric_mujoco.py`.

You can launch its main with `python3 Quadruped-PyMPC/inverse_kinematics_numeric_mujoco.py`.

1. Complete the code in the above file.

2. Start for a very far foot positions now (see its main): Does IK converge in one step to the right solution?
3. What happens if do multiple IK step?
4. What happens if you set the desired feet positions close to the singularity (search for comment EXERCISE 1.4 in the main)? Tune λ to converge the desired foot positions in less iteration than possible.

Exercise 2: Terrain Estimation

On uneven terrain, estimate a local terrain plane from the stance-foot positions. The terrain normal can then be used to generate a terrain-aligned desired base orientation. (check slide 12 in [Monday Locomotion](#)). This adaptation helps maintain a suitable leg configuration and improves contact consistency on slopes and uneven ground.

1. Run `python3 Quadruped-PyMPC/simulation/simulation.py` and walk over the stairs with the current version of the code. To do so, type `ictp` in the terminal and use the WASD keys.
2. Modify the main function of `Quadruped-PyMPC/quadruped_pympc/helpers/terrain_estimator_vertical_fit.py` and check the estimated terrain plane once you complete this part:

```
python3 Quadruped-PyMPC/quadruped_pympc/helpers/terrain_estimator_vertical_fit.py
```

3. Run `python3 simulation.py` again and walk over the stairs. To do so, type `ictp` in the terminal and use the WASD keys.
4. Introduce and tune a first order filter to avoid abrupt changes in the desired pitch and roll.

Exercise 3: MPC Tuning

The MPC should track desired CoM motion, body orientation, velocity, and contact forces. A typical cost function is:

$$J = \sum_{k=0}^{N-1} \left[\| \mathbf{x}_k - \mathbf{x}_k^{\text{ref}} \|_Q^2 + \| \mathbf{u}_k - \mathbf{u}_k^{\text{ref}} \|_R^2 + \| \mathbf{u}_k - \mathbf{u}_{k-1} \|_{R_\Delta}^2 \right] + \| \mathbf{x}_N - \mathbf{x}_N^{\text{ref}} \|_{Q_f}^2.$$

Important parameters are:

- Prediction horizon N ;
- MPC sampling time;
- Contact schedule and selected gait;
- CoM position and velocity weights;
- Base orientation and angular velocity weights;
- Contact-force regularization - we ask the GRF to be at least equal to the one needed to compensate gravity, e.g. not making the robot collapse. For example, for Go2 that weights 15kg, the sum of the GRFs in the z direction is at least $15 \cdot 9.81$ Nm. If the robot has 4 foot on the ground, each foot should at least give $15 \cdot 9.81 / 4$ Nm. In the x-y direction we regularize to 0, because we don't know them;

1. What happens if you set $W = 1000, 1000, 0.001$ at Grfs weights? or $W = 0.001, 0.001, 1000$? Modify the file `centroidal_nmpc_nominal.py`, (search for comment EXERCISE 3.1), and run `python3 Quadruped-PyMPC/simulation/simulation.py`.
2. Reduce to 1 the weight on pitch (base orientation weight) and try to climb on stairs.
3. Change the prediction horizon N and evaluate stability, tracking, and computational time. Evaluate especially the performances for the crawl gait. Modify the file `config.py`, line 92-215, and run `python3 Quadruped-PyMPC/simulation/simulation.py`.
4. Gradually increase the MPC loop duration dT_{mpc} from 0.002 up 0.2 and evaluate the impact on walking performances and computation time. Modify the file `config.py`, line 93, and run `python3 Quadruped-PyMPC/simulation/simulation.py`.
5. what is the best tradeoff between dT_{mpc} and MPC horizon N for the crawl? What is the drawback of having a longer horizon in a real setting?
6. Having an horizon allows the MPC to run at much lower frequencies than a reactive controller, what is the minimum frequency that the MPC can still stabilize the system? Modify the file `config.py`, N , dt , and run `python3 Quadruped-PyMPC/ros2/run_simulator.py` and in another terminal `python3 Quadruped-PyMPC/ros2/run_controller.py`
7. Compare the behaviour of a trot/crawl for different gait frequencies (from 1.4 Hz up to 2.4Hz) and check the different levels of robustness to disturbances and what the maximum velocity that you can achieve. What is the drawback of getting to high speed? Modify the file `config.py`, line 215-216, and run `python3 Quadruped-PyMPC/simulation/simulation.py`.

1 Exercise 4: State Estimation

Replace the perfect simulated state with an estimated state. The estimator combines:

- IMU angular velocity and linear acceleration;
- Joint encoders;
- Foot kinematics;
- Contact detection;
- Optional ground-truth state only for debugging.

An Extended Kalman Filter can estimate base orientation, position, velocity

For this, you need to run from Quadruped-PyMPC (first repo):

```
python3 Quadruped-PyMPC/ros2/run_simulator.py (TERMINAL 1)
python3 Quadruped-PyMPC/ros2/run_controller.py (TERMINAL 2)
```

and from the repo `iekf-legged-robot-pytorch` (second repo):

```
python3 ekf-legged-robot-pytorch/run_state_estimator_ros2.py (TERMINAL 3)
source ros2_ws/install/setup.bash (TERMINAL 4)
ros2 run plotjuggler plotjuggler (TERMINAL 4)
```

each of them with their own conda env!!!

1. Compare estimated and ground-truth base linear velocity:
topics: `base_state_debug`, `ekf_base_state_debug`.

2. Change the belief on the filter inputs changing the different noise. If you add bias on accelerometers what happens to the estimation?
Modify `python3 Quadruped-PyMPC/ros2/run_simulator.py` line 202. Change the noise in the filter settings to improve the estimation (Modify `config.py` inside `iekf-legged-robot-pytorch`).
3. Now remove the offset and try to add gaussian noise to the joint velocities and increase the appropriate filter covariance to mitigate that.
4. Study the effect of incorrect contact detection. Change the minimum force threshold for contact detection and see how estimator performances are degrading in position estimate, which direction is the most affected. (Modify `iekf-legged-robot-pytorch/config.py`).